CHIP-8

KOSS

CHIP-8 EMULATOR

This is a low level machine implementation of C++. This project features a custom terminal-based ASCII rendering engine, hardware-level collision detection, and a custom assembly-coded game environment designed for the KOSS Selection Task 2026.

MEMORY AND ARCHITECTURE

Memory Mapping: I implemented the 4096-byte RAM as a raw uint8_t array. The first 512 bytes are reserved for the system, so the ROM starts exactly at 0x200.

```
uint16_t pc=0x200;
uint16_t i=0;
uint8_t v[16] = {0};
uint8_t m[4096];
uint16_t s[16] = {0};
uint8_t sp = 0;
uint8_t display[64 * 32] = {0};
```

THINKING IN 4 BYTES

The Register File: Used 16 general-purpose registers (V0-Vf). The most interesting one to implement was VF, which I had to "wire" to double as a hardware flag for carry-bits and collision detection.

The Stack: Used a 16-level stack to handle nested subroutines ($CALL$ and RET opcodes), ensuring the Program Counter always knows where to return after a function ends.

FETCH-DECODE-EXECUTE LOOP

Bitwise Decoding: Since every opcode is 2 bytes, I used bit-masking and shifting to extract variables. For example, in an opcode like 0x8xy4, I used (opcode & 0x0F00) >> 8 to find the target register.

```
uint8_t  x   = (opcode & 0x0F00) >> 8;
uint8_t  y   = (opcode & 0x00F0) >> 4;
uint8_t  n   = (opcode & 0x000F);
uint8_t  kk  = (opcode & 0x00FF);
uint16_t nnn = (opcode & 0x0FFF);
```

The Heart of the Machinee

Emulation Speed: Instead of just letting the loop run at max CPU speed, I structured the while(true) loop to handle one 16-bit instruction per cycle, effectively "clocking" the virtual CPU.

Opcode Diversity: I implemented 30+ opcodes, covering everything from basic jumps to the complex $CXNN$ random number generator.

Custom Rendering Engine

Why the Terminal? I wanted a pure C++ implementation without forcing the user to install external libraries like SDL or OpenGL.

```
if (spritePixel != 0) {
    if (display[screenPixelIndex] == 1) {
        v[0xF] = 1;
    }
    display[screenPixelIndex] ^= 1;
}
```

Zero-Dependency Graphics

ASCII Mapping: I mapped the 64x32 monochrome grid to the Windows console. Active pixels are rendered as block characters (\xDB\xDB) and empty ones as spaces.

XOR Logic: The hardest part was the $DXYN$ instruction. CHIP-8 pixels aren't just "drawn"—they are XORed. If a sprite is drawn over an existing pixel, it flips. This hardware quirk is exactly how collision detection is handled natively.

Assembly Project: "Dot Hunter"

```
// CUSTOM KOSS GAME: "Dot Hunter" (WASD to catch the random teleporting dot)
uint8_t customGame[] = {
    0x60, 0x20,
    0x61, 0x10,
    0xC2, 0x3F,
    0xC3, 0x1F,
    0xA2, 0x34,

    0x00, 0xE0,
    0xD2, 0x31,
    0xD0, 0x11,

    0x3F, 0x01,
    0x12, 0x18,
    0xC2, 0x3F,
    0xC3, 0x1F,

    0xF4, 0x0A,

    0x34, 0x04,
    0x12, 0x20,
    0x70, 0xFF,

    0x34, 0x06,
    0x12, 0x26,
    0x70, 0x01,

    0x34, 0x02,
    0x12, 0x2C,
    0x71, 0xFF,

    0x34, 0x08,
    0x12, 0x32,
    0x71, 0x01,

    0x12, 0x0A,

    0x80
};
```

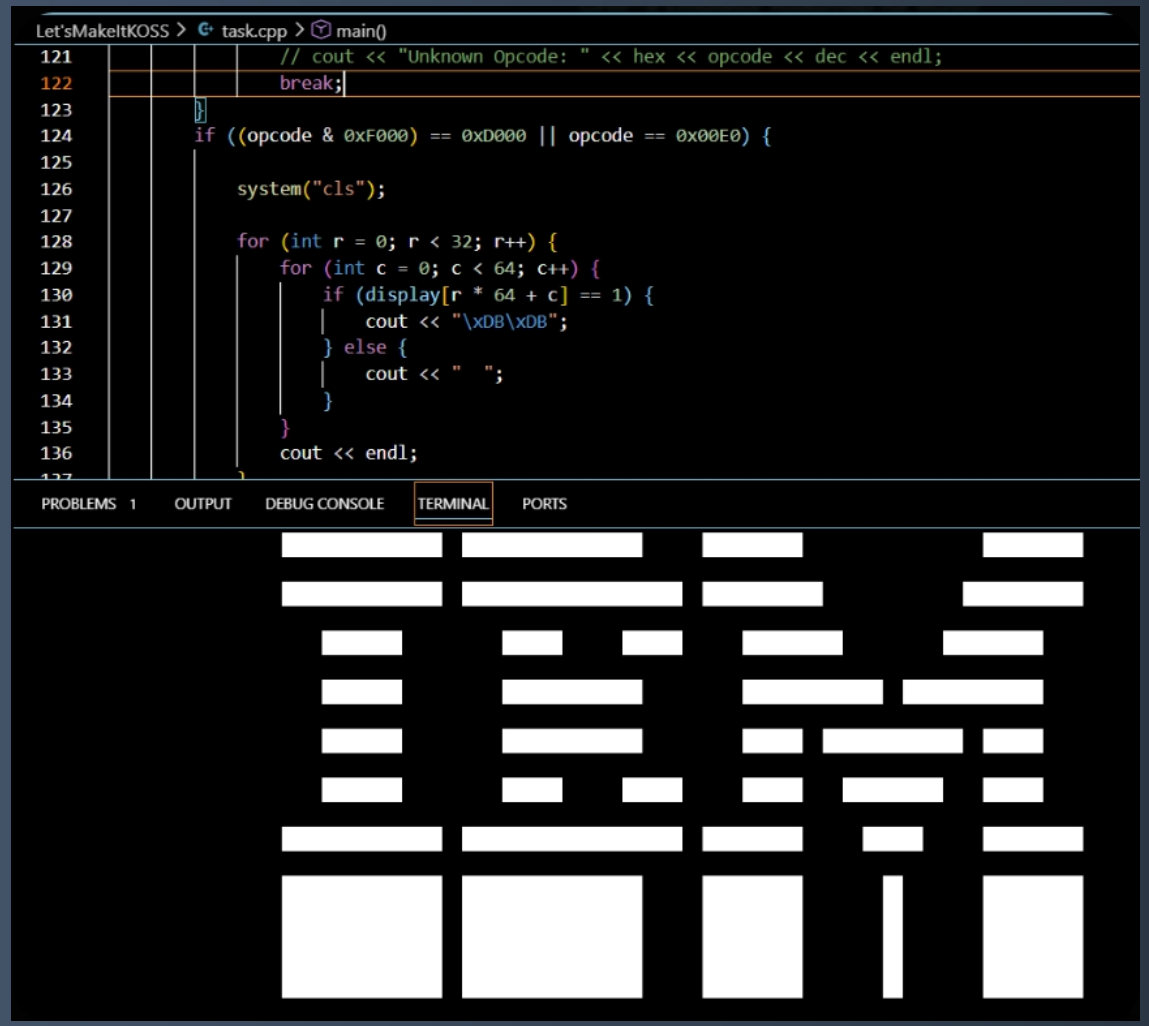
Programming in Raw Hex

The Concept: I wrote a custom game in raw assembly to stress-test my emulator's input and collision logic.

Instruction Chaining: Since there are no if-else blocks in CHIP-8, I built a 4-way controller (WASD) by chaining 3XKK (Skip if Equal) instructions.

Hardware Physics: The coin-catch logic doesn't use any coordinate math. It simply monitors the Vf register. When the player sprite XORs with the coin sprite, the Vf flag is triggered by the CPU, and the game teleports the coin to a new random coordinate.

CONCLUSION AND FUTURE POSSIBILITIES



Closing Thoughts

Learning Outcomes: This task gave me a deep dive into bitwise manipulation, memory-mapped I/O, and the fundamental Fetch-Execute cycle that powers every modern processor

Next Steps: Given more time, I plan to implement the 60Hz hardware timers (Delay and Sound) and add a more robust file-picker for external .ch8 ROMs.

Source Code: My full implementation is clean, commented, and ready for review.

CHIP-8

THANKS
FOR READING